Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la lucrarea de laborator nr. 6

Disciplina: Tehnici de programare

Tema: Informații invizibile forensics și steganografie

A efectuat: Moraru Elena, grupa SI-264

A verificat: asistent universitar Reițman Patricia

Chișinău 2026

Organizarea lucrării

Scopul lucrării este identificarea informațiilor ascunse prin codări, XOR, hashuri, semnături de fișiere, metadate și steganografie LSB.

decode.py acoperă A1–A7, iar partea2.py acoperă B1–B6.
Scriptul furnizat în fișă generează fișierele de intrare în directorul probe.

Fișierele predate

```
arsenal/forensics/decode.py
arsenal/forensics/partea2.py
arsenal/forensics/fisa.json
arsenal/forensics/stego.png
teste/test_decode.py
probe/pregateste.py
Raport_Laboratorul_6.pdf
```

Rulare din folderul lab 6

```
python -m pip install Pillow
python probe/pregateste.py
python arsenal/forensics/decode.py U2FsdXQ=
python arsenal/forensics/partea2.py
python teste/test_decode.py
```

Programul produce fisa.json, gasit.zip și stego.png în arsenal/forensics.
Pentru rularea unui singur exercițiu se folosește --pas, de exemplu --pas A5.
Pentru B6 se rulează mai întâi B5.

A1. Text și octeți

Transformăm Salut în octeți, afișăm valoarea primului octet și forma hexazecimală. Reconversia produce același text.

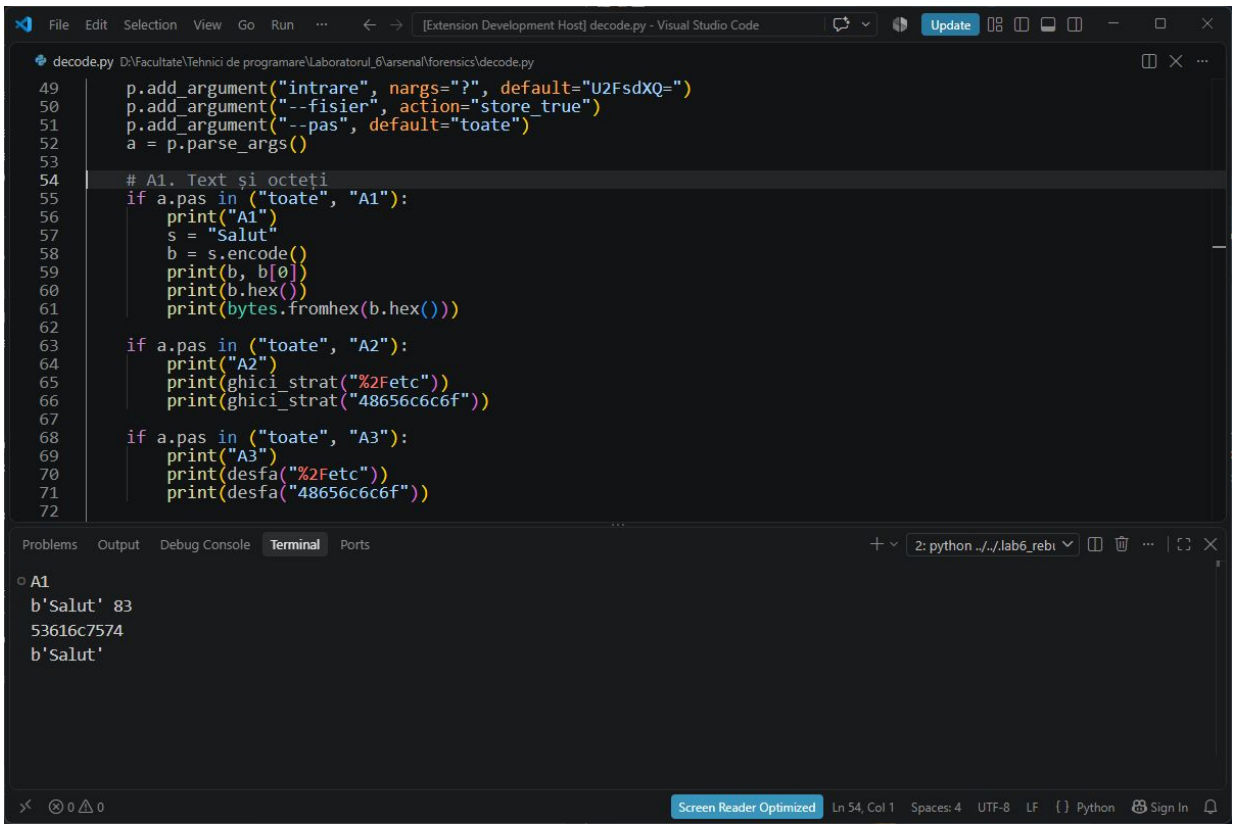


Fig. 1.1. Conversia textului și rezultatul rulării.

A2. Recunoașterea codării

Funcția ghici_strat verifică forma șirului. Exemplele %2Fetc și 48656c6c6f sunt recunoscute ca URL, respectiv hex.

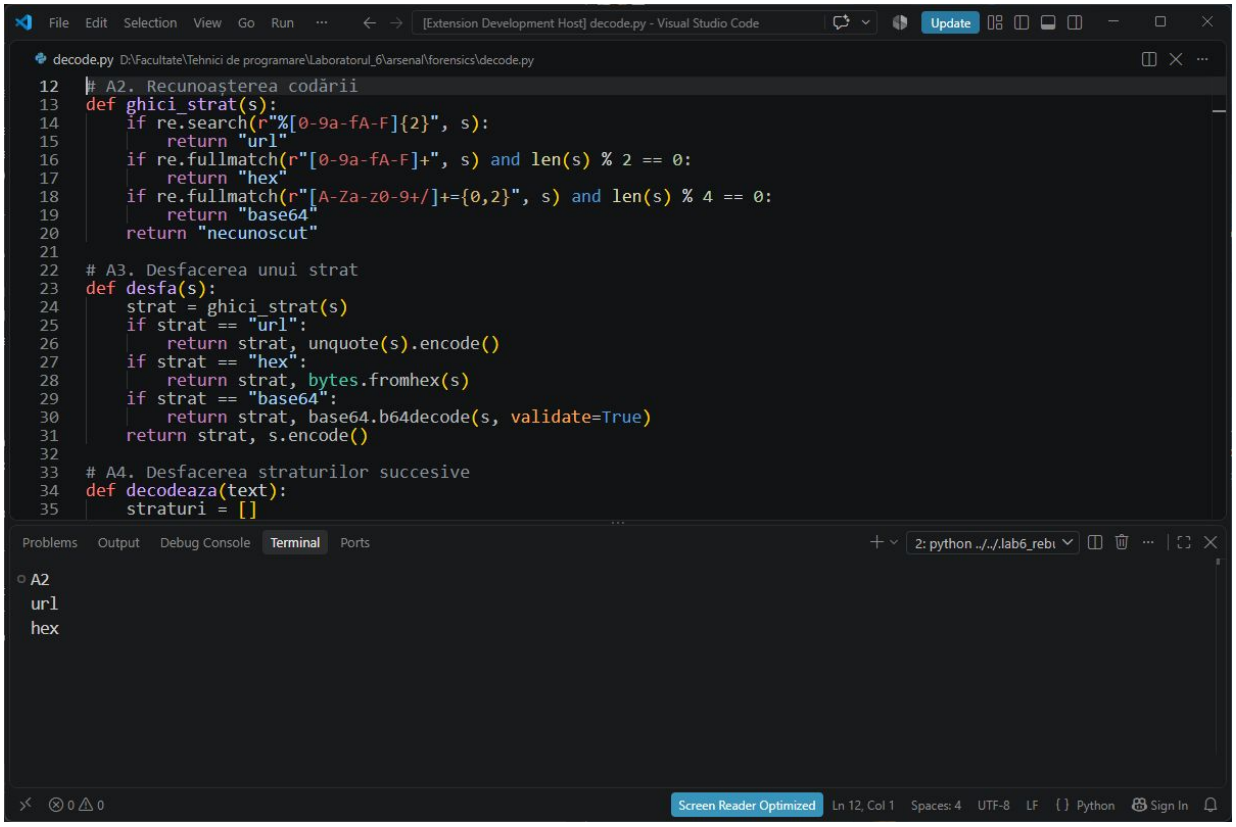


Fig. 1.2. Identificarea tipului de codare.

A3. Desfacerea unui strat

Funcția desfa aplică decodarea potrivită și întoarce tipul stratului împreună cu octeții obținuți: /etc și Hello.

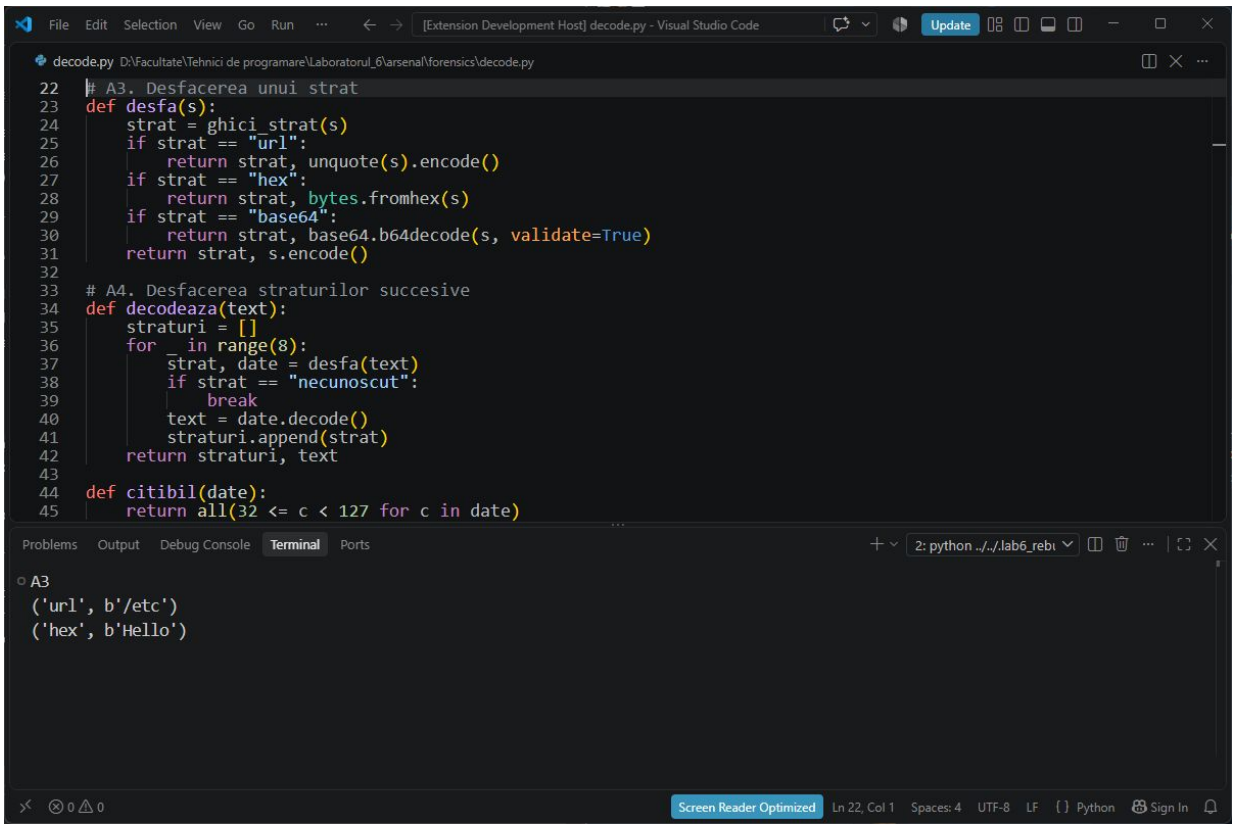


Fig. 1.3. Decodarea URL și hex.

A4. Decodarea succesivă

Repetăm decodarea cel mult opt ori. Ne oprim când șirul nu mai are forma unei codări recunoscute.

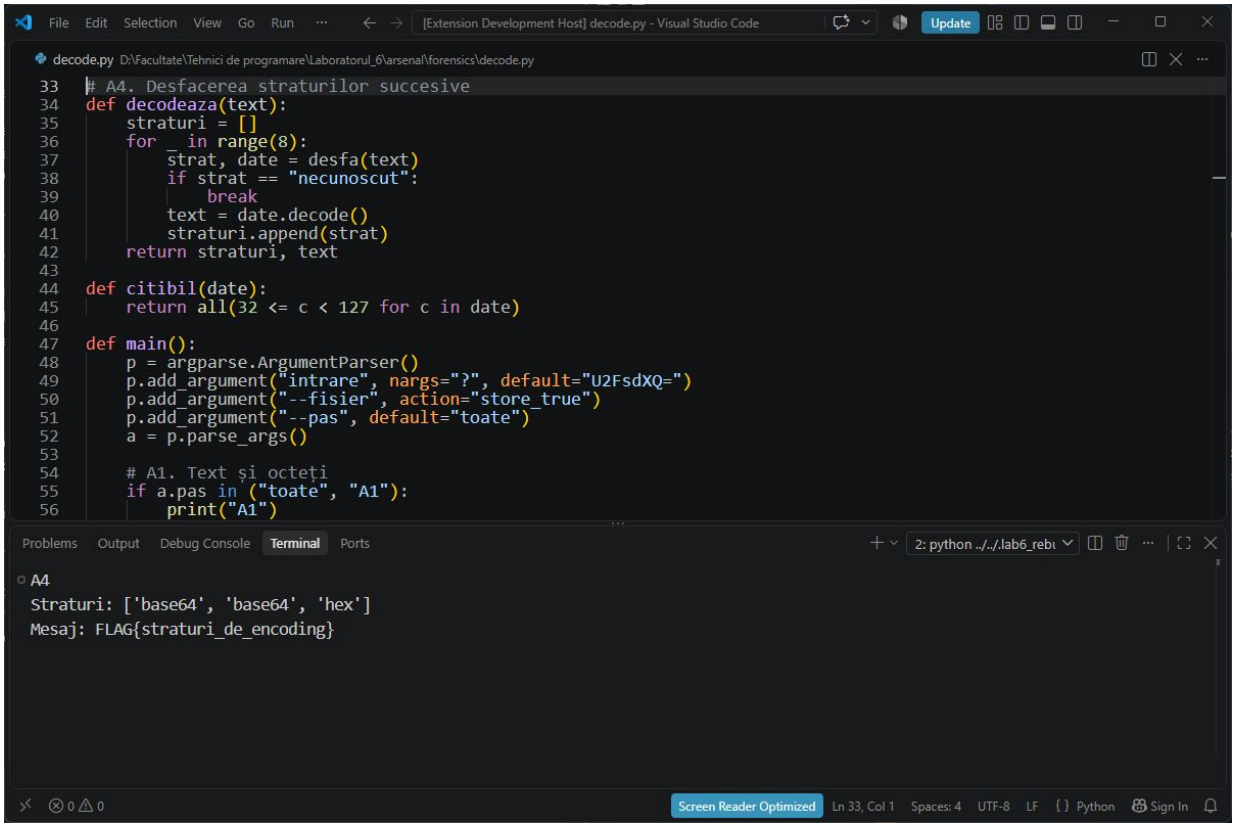


Fig. 1.4. Funcția de decodare și rularea A4.

A4. Citirea mesajului din fișier

Din mesaj.txt se elimină straturile base64, base64 și hex. Rezultatul este FLAG{straturi_de_encoding}.

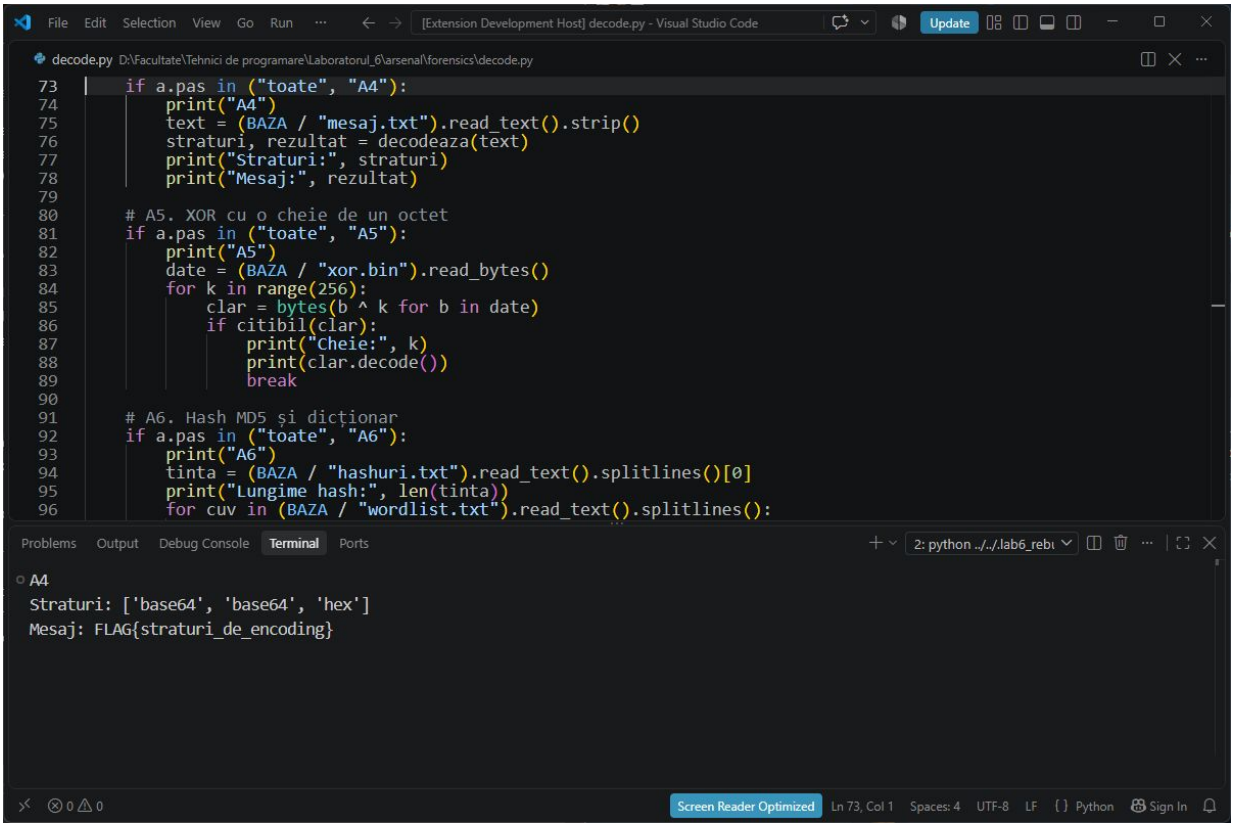


Fig. 1.4 (continuare). Apelarea funcției pentru mesaj.txt.

A5. XOR cu un octet

Încercăm cheile 0-255 și verificăm dacă rezultatul este imprimabil. Pentru proba dată, cheia 128 produce mesajul corect.

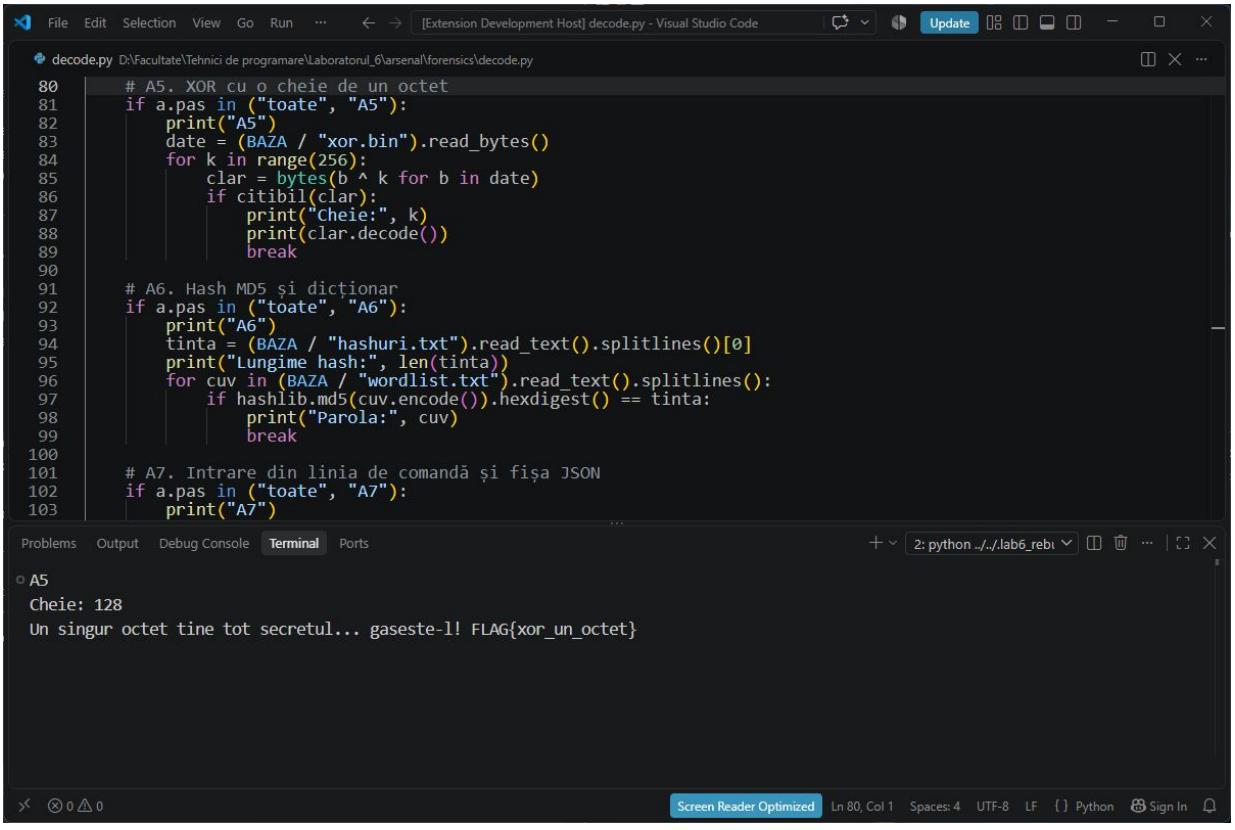


Fig. 1.5. Cheia XOR și mesajul recuperat.

A6. Compararea hashurilor

Primul hash are 32 de caractere. Calculăm MD5 pentru fiecare cuvânt din dicționar; password are același hash.

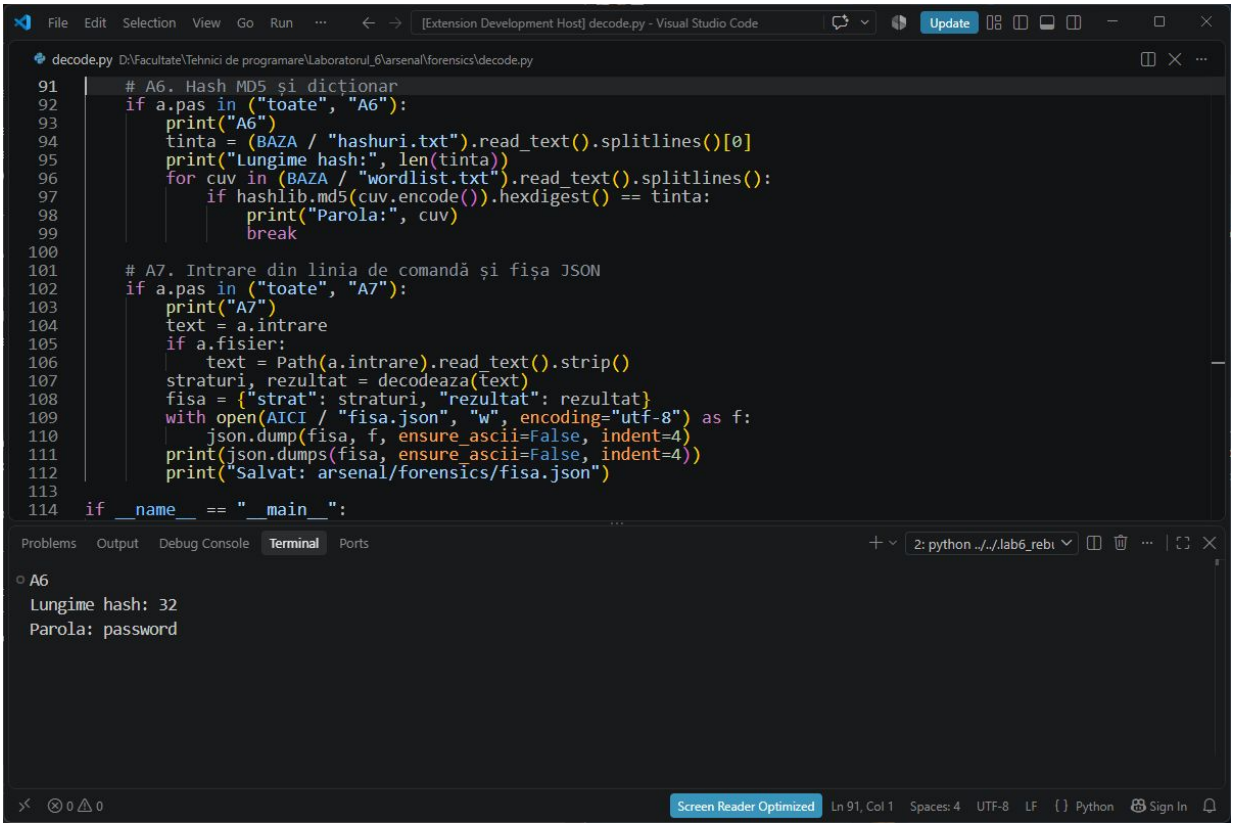


Fig. 1.6. Căutarea prin dicționar pentru primul hash MD5.

A7. Argumentele programului

Argumentul intrare primește un șir; --fisier permite citirea dintr-un fișier. --pas selectează un singur exercițiu.

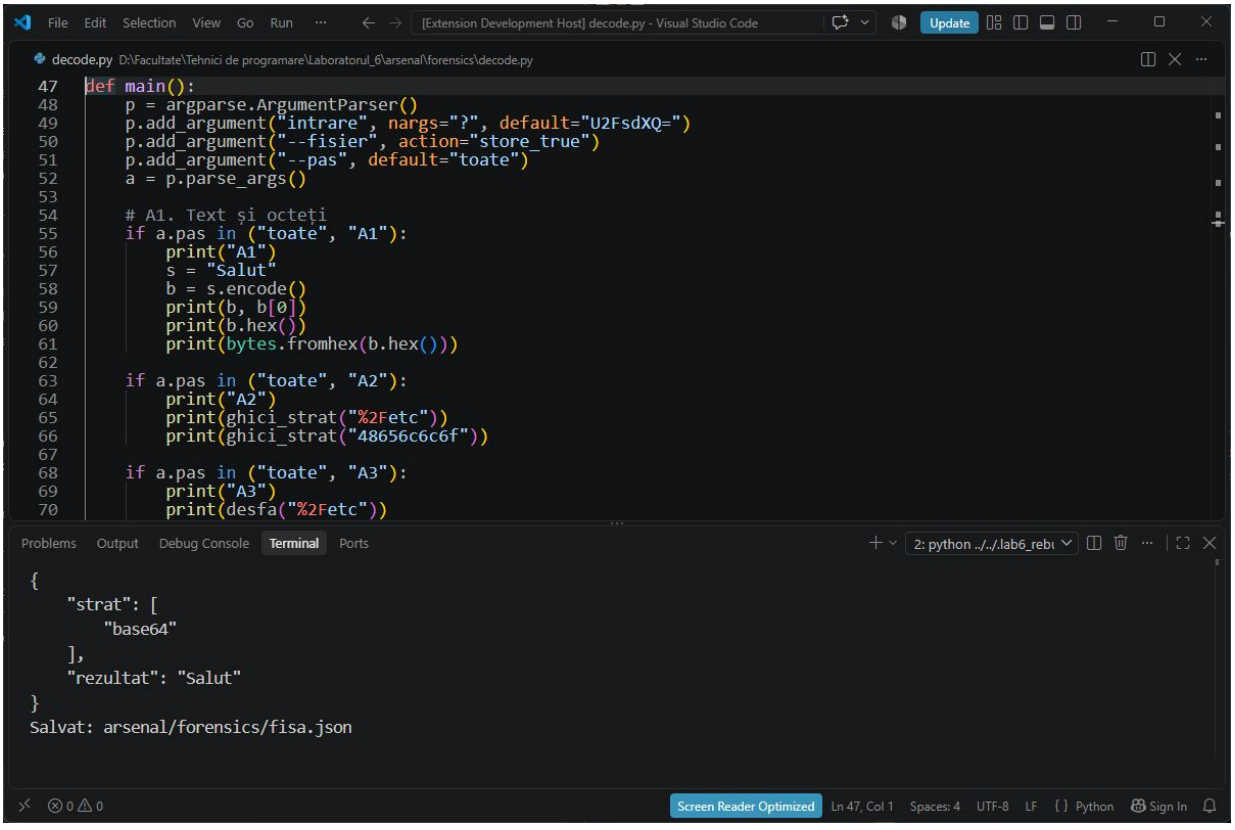


Fig. 1.7. Argumentele și rezultatul rulării A7.

A7. Salvarea fișei JSON

Salvăm straturile și rezultatul în fisa.json, lângă decode.py. Pentru U2FsdXQ= obținem un strat base64 și textul Salut.

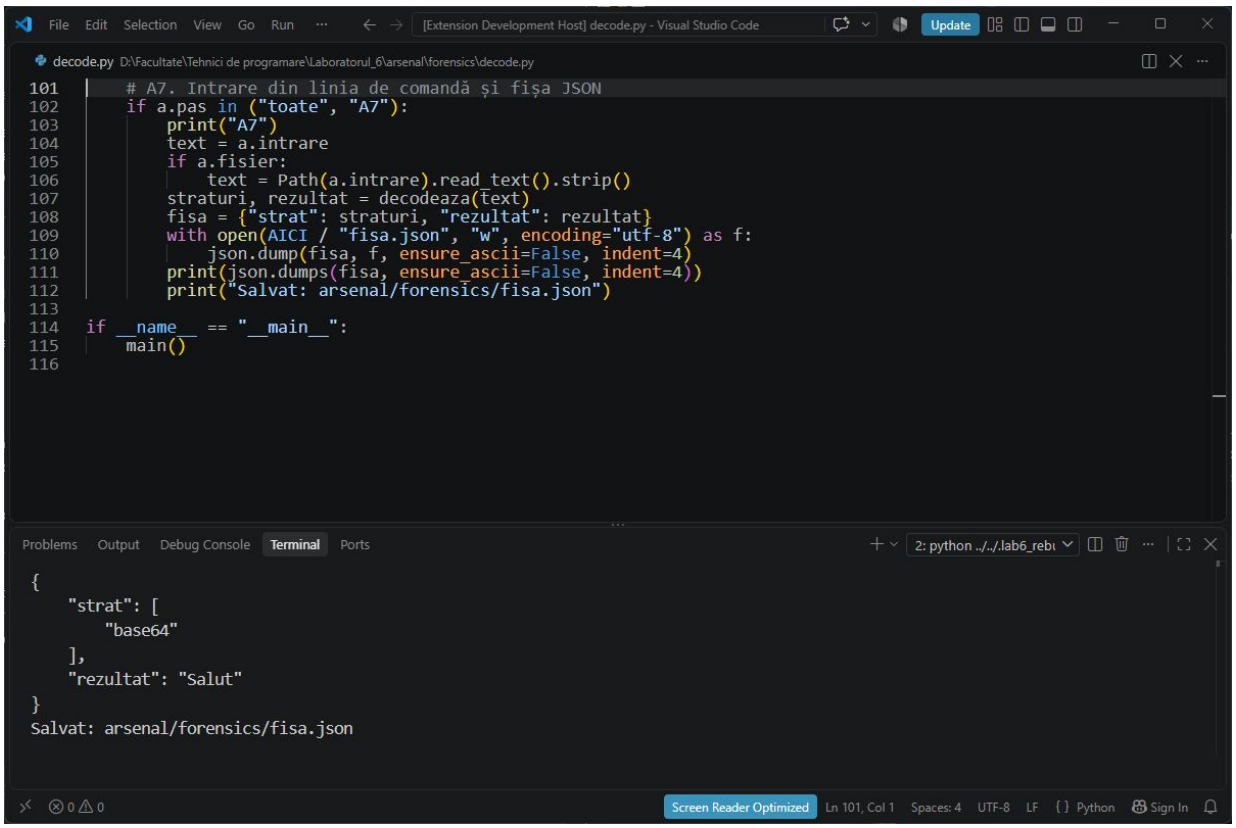


Fig. 1.7 (continuare). Crearea fișierului fisa.json.

Verificarea funcției desfa

Testul verifică tipul base64 și octeții b'Salut'. La rulare, mesajul test_base64: OK confirmă că verificarea a trecut.

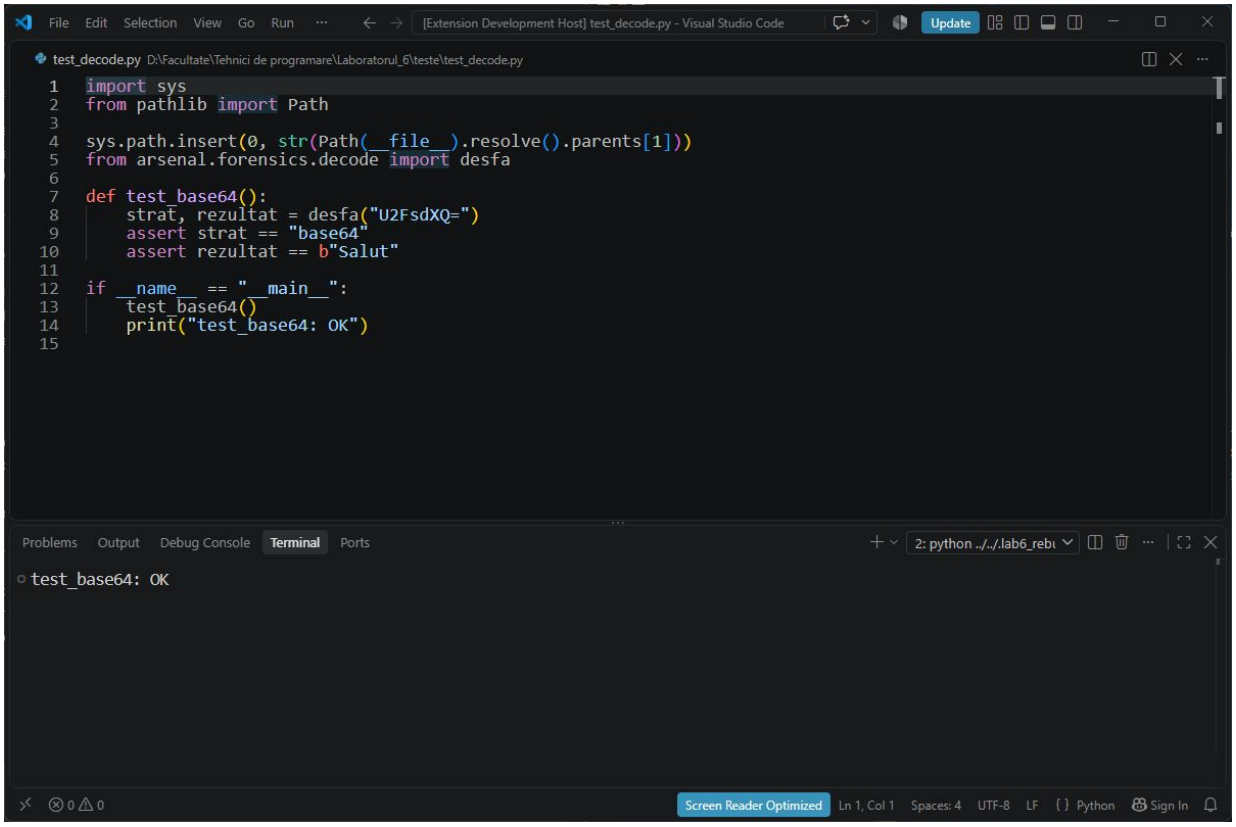


Fig. 1.8. Testul automat și rularea lui.

B1. Identificarea tipului real

Citim primii opt octeți și comparăm semnăturile cunoscute. Probele foto.jpg și ascuns.png sunt identificate drept JPG și PNG.

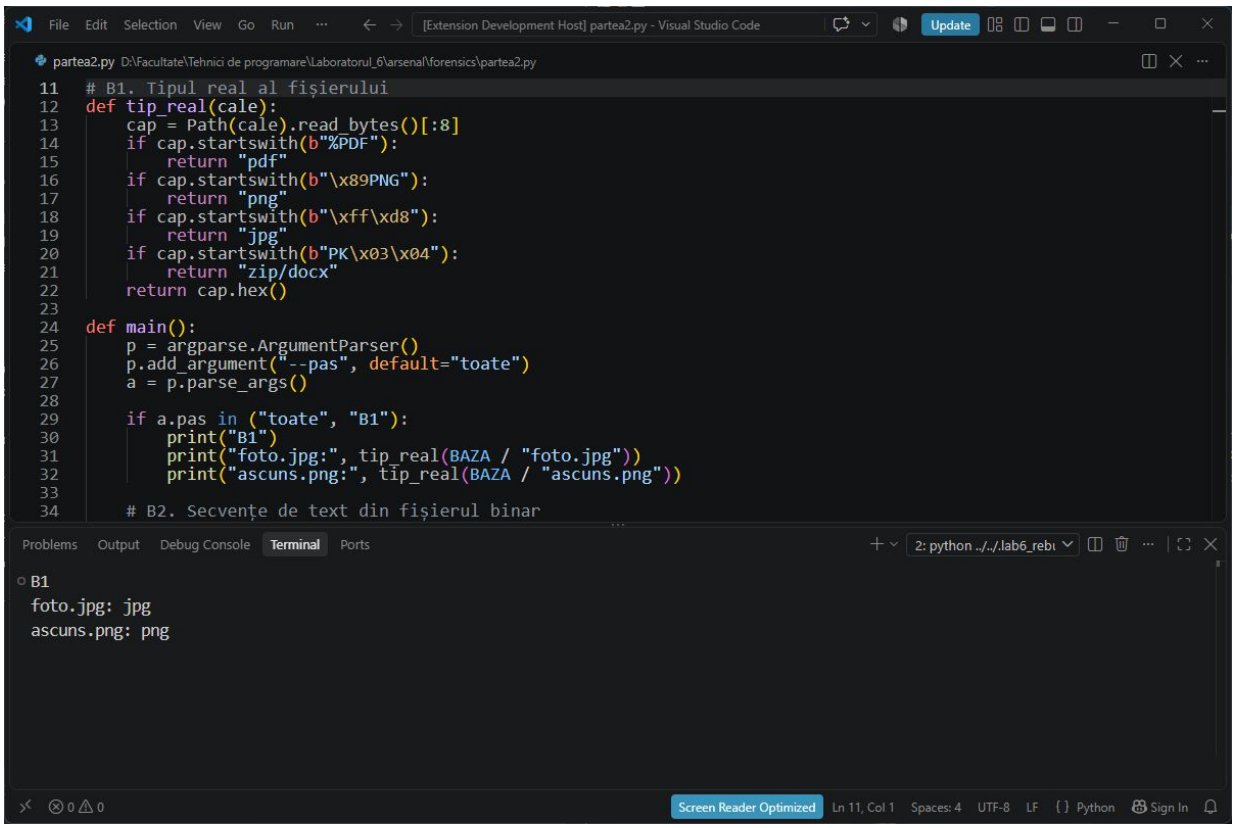


Fig. 2.1. Identificarea fișierelor după semnătură.

B2. Extragerea șirurilor citibile

Căutăm secvențe de cel puțin patru caractere ASCII imprimabile. În ascuns.png apar secret.txt și FLAG{ascuns_prin_carving}.

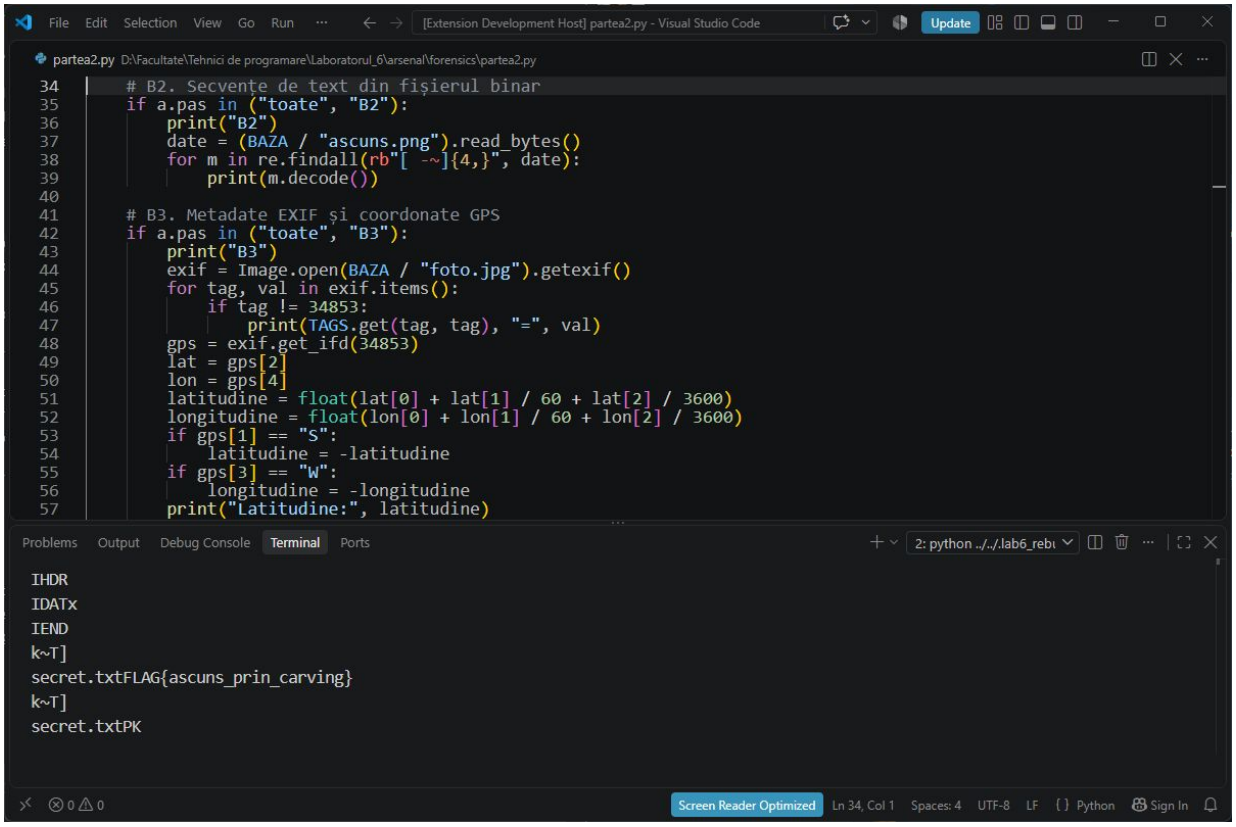


Fig. 2.2. Șirurile citibile găsite în fișier.

B3. Metadate EXIF și coordonate

Citim metadatele probei generate: Apple, iPhone 13 și data. Transformăm GPS din grade, minute, secunde în 47.016667 și 28.8575.

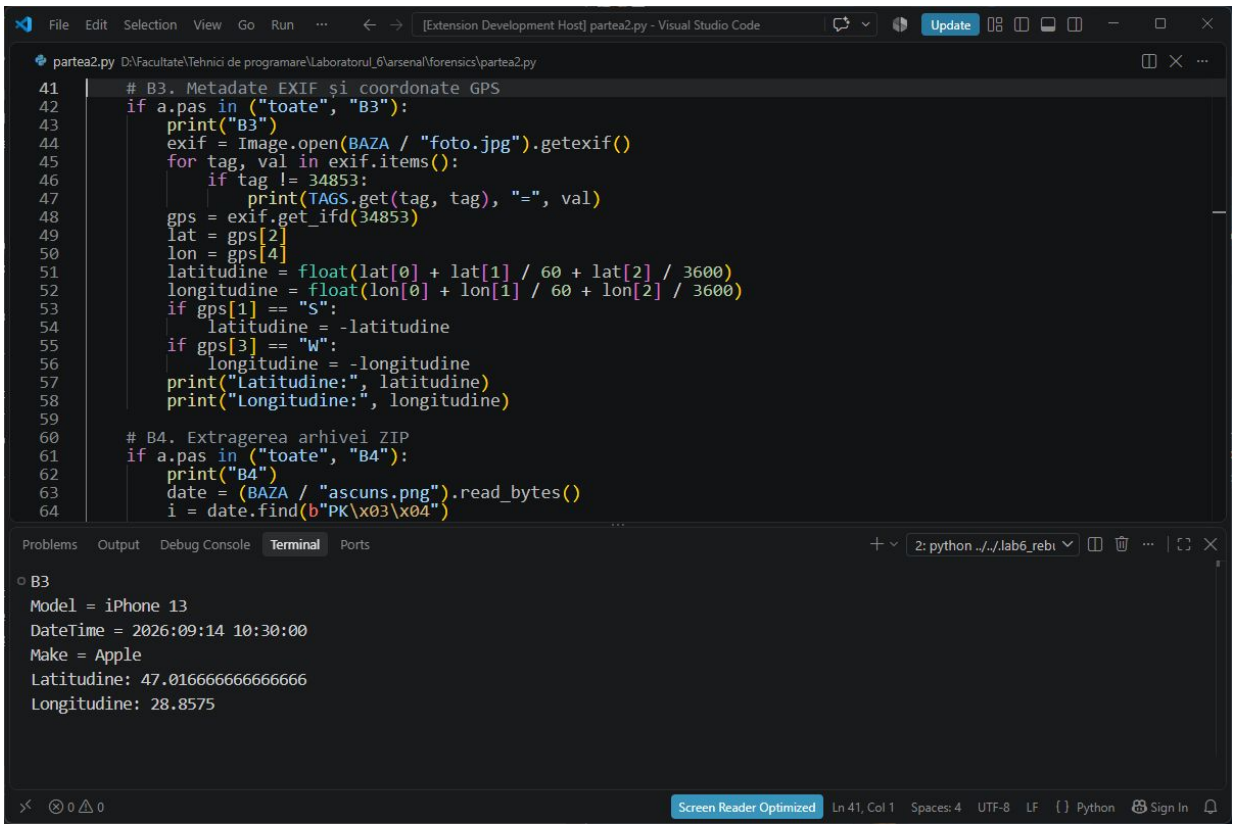


Fig. 2.3. Metadatele de test și coordonatele GPS.

B4. Extragerea arhivei ascunse

Semnătura ZIP apare de la octetul 782. Salvăm datele în gasit.zip și citim secret.txt: FLAG{ascuns_prin_carving}.

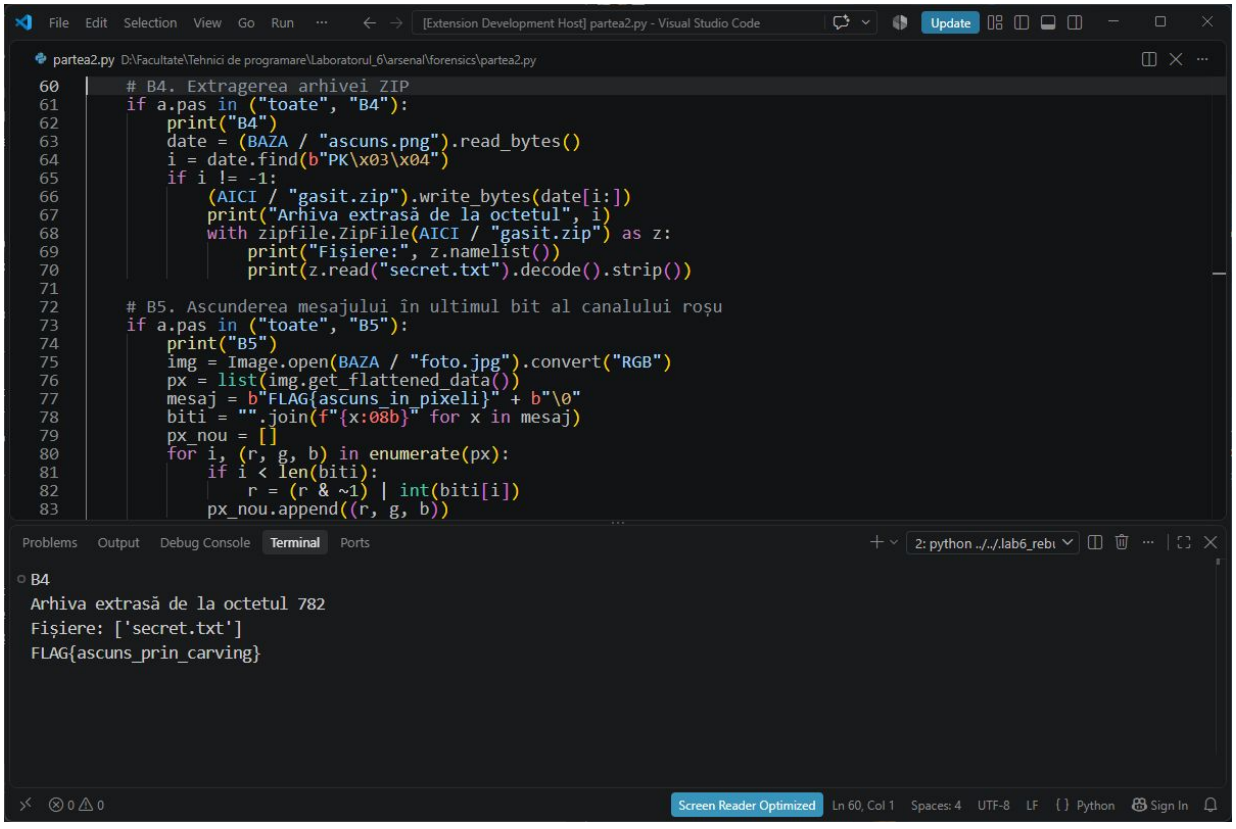


Fig. 2.4. Extragerea arhivei și verificarea conținutului.

B5. Ascunderea mesajului în pixeli

Scriem biții mesajului în bitul cel mai puțin semnificativ al canalului roșu. Adăugăm un octet zero de oprire: în total 184 biți.

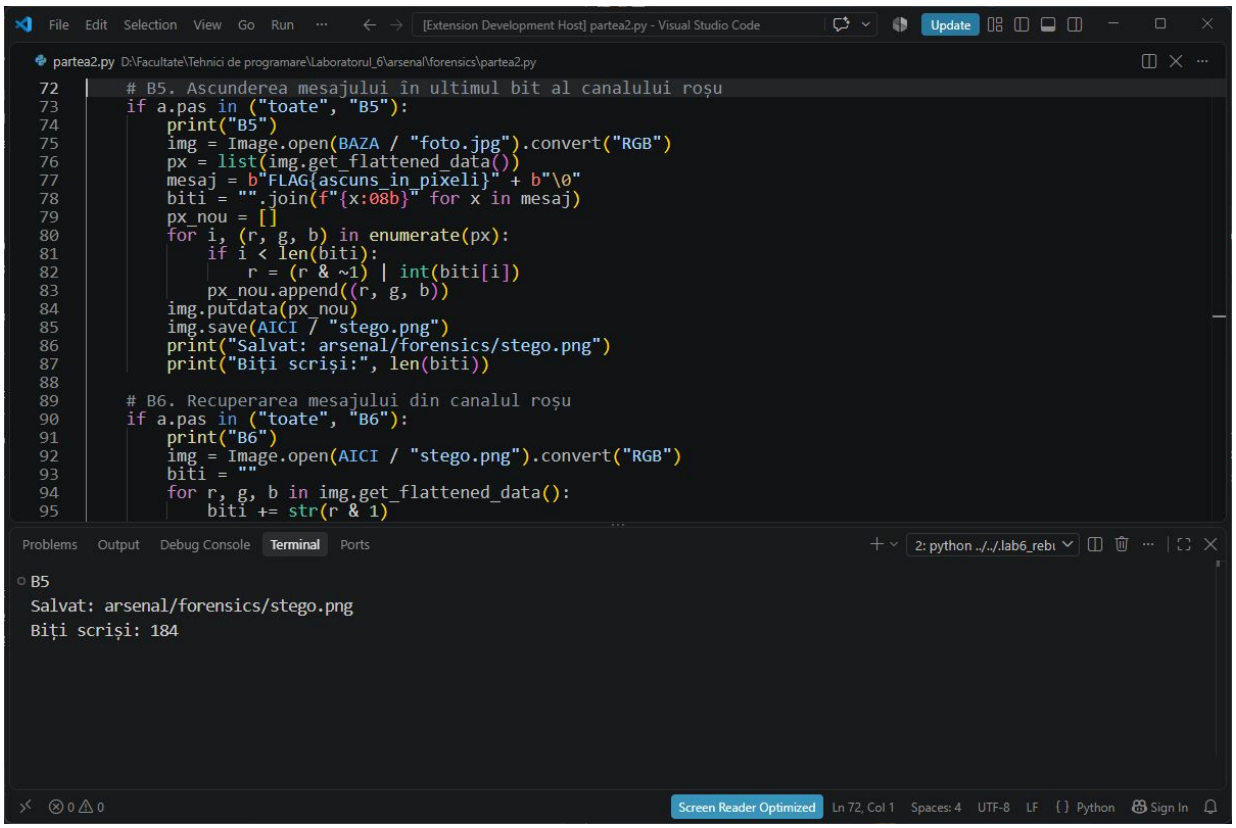


Fig. 2.5. Salvarea imaginii stego.png.

B6. Recuperarea mesajului ascuns

Citim aceiași biți, îi grupăm câte opt și ne oprim la octetul zero. Recuperăm exact FLAG{ascuns_in_pixeli}.

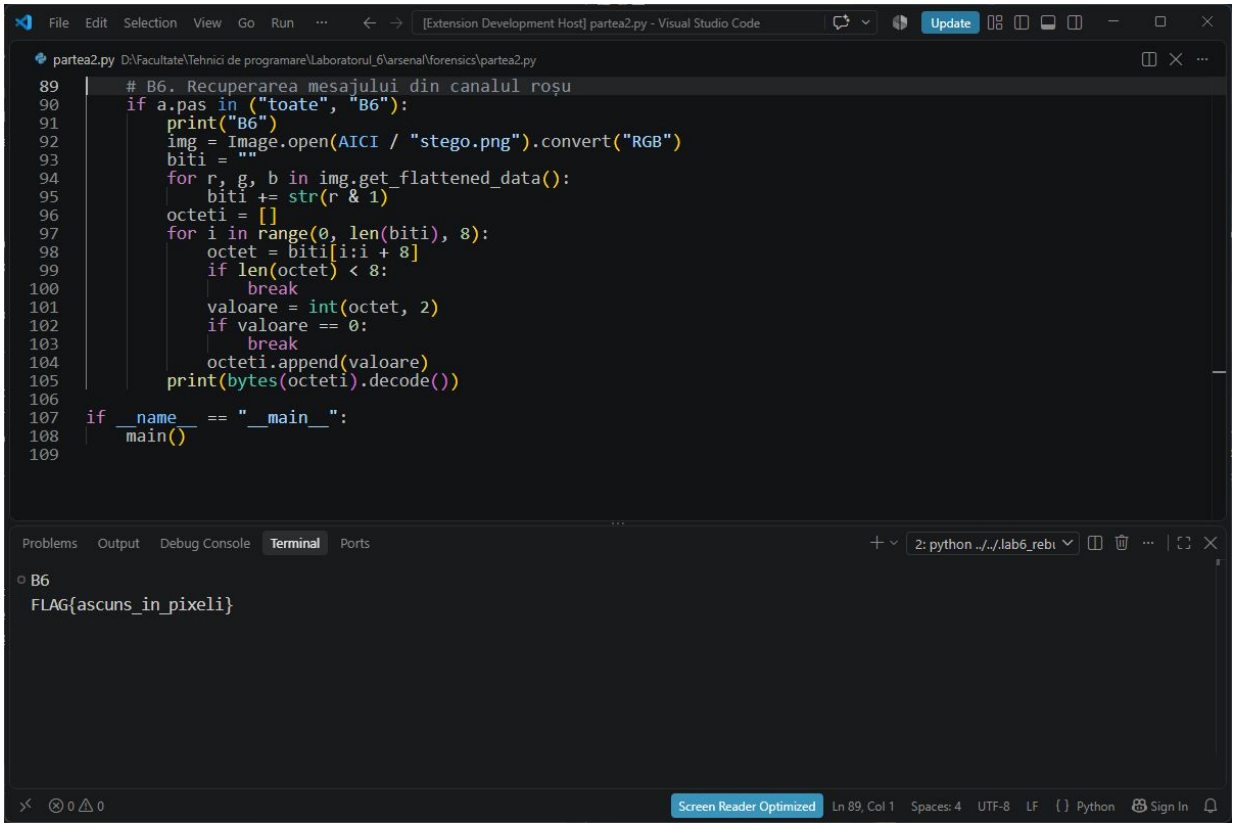


Fig. 2.6. Extragerea mesajului din stego.png.

Concluzie: am recuperat mesajele prin decodare, XOR, carving și LSB. Hashul a fost verificat prin comparație, iar metadatele au oferit informații despre proba analizată.